

Лабораторная работа №2

Тема работы: дружественные функции и классы, перегрузка операторов.

Цель работы: Понять назначение дружественных функций и классов, изучить принципы перегрузки бинарных и унарных операций.

Теоретические сведения: рассмотрена в соответствующих разделах [1, 3–8].

Дружественные функции. Иногда возникает необходимость организации доступа к локальным данным нескольких классов из одной функции. Для реализации этого в C++ введен спецификатор `friend`. Если некоторая функция определена как `friend`-функция для некоторого класса, то эта функция называется дружественной и она:

- не является компонентом-функцией этого класса;
- имеет доступ ко всем компонентам этого класса (`private`, `public` и `protected`).

Пример использования дружественной функции.

```
#include <iostream>
using namespace std;

class kls
{
    int i,j;
public:
    kls(int I,int J) : i(I),j(J) {}           // конструктор
    int max() {return i>j? i : j;}           // функция-компонент класса kls
    friend double fun(int, kls&);           // friend-объявление внешней функции fun
};
double fun(int i, kls &x)                   // описание дружественной функции
{
    return (double)i/x.i;
}
int main()
{
    kls obj(2,3);                           // объявление объекта
    cout << obj.max() << endl;
    cout << fun(3,obj) << endl;             // вызов дружественной функции
    return 0;
}
```

Функции со спецификатором `friend`, не являясь компонентами класса, не имеют и, следовательно, не могут использовать `this` указатель. Следует также отметить ошибочность следующей заголовочной записи функции

```
double kls :: fun(int i,int j),
```

т. к. `fun` не является компонентом-функцией класса `kls`.

В общем случае `friend`-функция является глобальной независимо от секции, в которой она объявлена (`public`, `protected`, `private`), при условии, что она не

объявлена ни в одном другом классе без спецификатора friend. Функция friend, объявленная в классе, может рассматриваться как часть интерфейса класса с внешней средой.

Вызов компоненты-функции класса осуществляется с использованием операции доступа к компоненте «.» или «->». Вызов же friend-функции производится по ее имени. В friend-функцию не передается this-указатель и доступ к компонентам класса выполняется либо явно «.», либо косвенно «->».

Компонент-функция одного класса может быть объявлена со спецификатором friend для другого класса:

```
class X{ .....  
    void fun (....);  
};  
class Y{ .....  
    friend void X:: fun (....);  
};
```

В приведенном фрагменте функция fun() имеет доступ к локальным компонентам класса Y. Запись вида friend void X:: fun (....) говорит о том, что функция fun принадлежит классу X, а спецификатор friend разрешает доступ к локальным компонентам класса Y (т. к. она объявлена со спецификатором в классе Y).

Ниже приведен пример программы расчета суммы двух налогов на зарплату. Используется дружественная функция.

```
#include <iostream>  
#include <string>  
#include <iomanip>  
using namespace std;  
class nalogi;           // неполное объявление класса nalogi  
class work  
{  
    char s[20];         // фамилия работника  
    int zp;             // зарплата  
public:  
    float raschet(nalogi); // компонент-функция класса work  
    void input()  
    {  
        cout << "вводите фамилию и зарплату" << endl;  
        cin >> s >> zp;  
    }  
    work(){}  
    ~work(){};  
};  
class nalogi  
{  
    float pd,           // подоходный налог  
    st;                 // налог на социальное страхование
```

```

    friend float work::raschet(nalogi); // friend-функция класса nalogi
public:
    nalogi(float f1,float f2) : pd(f1),st(f2){};
    ~nalogi(void){};
};
float work::raschet(nalogi nl)
{
    cout << s << setw(6) << zp << endl;      // доступ к данным класса work
    cout << nl.pd << setw(8) << nl.st << endl; // доступ к данным класса nalogi
    return zp*nl.pd/100+zp*nl.st/100;
}
int main()
{
    setlocale(LC_ALL,"Russian");
    nalogi nlg((float)12,(float)2.3); // описание и инициализация объекта
    work wr[2];                       // описание массива объектов
    for(int i=0;i<2;i++)
    {
        wr[i].input();                // инициализация массива объектов
    }
    cout << setiosflags(ios::fixed) << setprecision(3) << endl;
    cout << wr[0].raschet(nlg) << endl;    // расчет налога для объекта wr[0]
    cout << wr[1].raschet(nlg) << endl;    // расчет налога для объекта wr[1]
    return 0;
}

```

Следует отметить необходимость выполнения неполного (предварительного) объявления класса `nalogi`, т. к. в прототипе функции `raschet` класса `work` используется объект класса `nalogi`, объявляемого далее. Все функции одного класса можно объявить со спецификатором `friend` по отношению к другому классу следующим образом.

```

class X{ .....
    friend class Y;
    .....
};
class Y{ .....
};

```

В этом случае все компоненты-функции класса `Y` имеют спецификатор `friend` для класса `X` (имеют доступ к компонентам класса `X`). Класс `Y` является дружественным классу `X`.

В приведенном ниже примере оба класса являются дружественными.

```

#include <iostream>
using namespace std;
class B;
class A

```

```

{
    int i;          // компонент-данные класса A
public:
    friend class B; // объявление класса B другом класса A
    A():i(1){}      // конструктор
    ~A(){}          // деструктор
    void f1_A(B &); // метод, оперирующий данными обоих классов
};
class B
{
    int j;          // компонент-данные класса B
public:
    friend A;       // объявление класса A другом класса B
    B():j(2){}      // конструктор
    ~B(){}          // деструктор
    void f1_B(A &a) // метод, оперирующий данными обоих классов
    {
        cout << a.i + j << endl;
    }
};
void A :: f1_A(B &b)
{
    cout << i << ' ' << b.j << endl;
}
int main()
{
    A aa;
    B bb;
    aa.f1_A(bb);
    bb.f1_B(aa);
}

```

Результат выполнения программы:

```

1 2
3

```

В объявлении класса A содержится инструкция `friend class B`, являющаяся и предварительным объявлением класса B и объявлением класса B дружественным классу A. Отметим также необходимость описания функции `f1_A` после явного объявления класса B (в противном случае не может быть создан объект b еще не объявленного типа).

Отметим основные свойства и правила использования спецификатора `friend`:

- `friend`-функции не являются компонентами класса, но имеют доступ ко всем его компонентам независимо от их атрибута доступа;
- `friend`-функции не имеют указателя `this`;
- `friend`-функции не наследуются в производных классах;

- отношение friend не является *ни симметричным* (т. е. если класс A friend классу B, то это не означает, что B также является friend классу A), *ни транзитивным* (т. е. если A friend B и B friend C, то не следует, что A friend C);
- друзьями класса можно определить перегруженные функции. Каждая перегруженная функция, используемая как friend для некоторого класса, должна быть явно объявлена в классе со спецификатором friend.

Перегрузка операторов.

Программы на языке C++ используют некоторые ранее определенные простейшие классы (типы), такие, как int, char, float и т. д. Мы можем описать объекты указанных классов, например:

```
int a,b;
char c,d,e;
float f;
```

Здесь переменные a, b, c, d, e, f можно рассматривать как простейшие объекты. В языке определены множество операций над простейшими объектами, выражаемых через операторы, такие, как «+», «-», «*», «/», «%» и т. д. Каждый оператор можно применить к операндам определенного типа.

```
float a, b=3.123, c=6.871;
a=c+b;      // нет ошибки
a=c%b;      // ошибка
```

Второе является ошибочным, поскольку оператор «%» должен быть приложен лишь к объектам целого типа. Из этого следует, что операторы языка можно применить к тем объектам, для которых они были определены.

К сожалению, лишь определенное число типов непосредственно поддерживается любым языком программирования. Однако многие операторы можно определить через классы в C++. Рассмотрим пример.

Пусть заданы множества A и B:

$A=\{a_1, a_2, a_3\}$; $B=\{a_3, a_4, a_5\}$;

Необходимо выполнить операции пересечение множеств «&» и объединение множеств «|»:

$A \& B = \{a_1, a_2, a_3\} \& \{a_3, a_4, a_5\} = \{a_3\}$;
 $A | B = \{a_1, a_2, a_3\} | \{a_3, a_4, a_5\} = \{a_1, a_2, a_3, a_4, a_5\}$;

Языки C/C++ не поддерживают непосредственно эти операции, однако в языке C++ можно объявить класс, назвав его set (множество). Далее можно определить операции над этим классом, выразив их с помощью знаков операторов, которые уже есть в языке C++, например «&» и «|». В результате операции «&» и «|» можно будет использовать как и раньше, а также снабдить их дополнительными функциями (объединения и пересечения множеств). Как определить, какую функцию должен выполнять оператор: старую или новую? Надо посмотреть на тип операндов в соответствующем выражении. Если операнды – это объекты целого типа, то нужно выполнить операцию

«побитового И» или «побитового ИЛИ». Если же операнды – это объекты типа `set`, то надо выполнить объединение или пересечение соответствующих множеств.

Функция `operator`. Функция `operator` может быть использована для расширения области приложения следующих операторов: «+», «-», «*», «%», «&», «|» и т.д. **Операторы, которые не могут быть перегружены:** «.», «.*», «::», «?:», `sizeof` и `typeid`.

Для перегрузки (доопределения) оператора разрабатываются функции, являющиеся либо компонентами, либо `friend`-функциями того класса, для которого они используются. Для того, чтобы перегрузить оператор, требуется определить действие этой оператора внутри класса. Общая форма записи функции-оператора являющейся компонентой класса, имеет вид

```
тип_возв_значения имя_класса::operator#(список аргументов)
{
    действия, выполняемые применительно к классу
}
```

Вместо символа «#» ставится значок перегружаемого оператора. Оператор всегда определяется по отношению к компонентам некоторого класса. В результате ее старое предназначение сохраняет силу. Функция `operator` является компонентом класса. При этом в случае унарного оператора функция `operator` не будет иметь аргументов, а в случае бинарного оператора будет иметь один аргумент. В качестве отсутствующего аргумента используется указатель `this` на тот объект, в котором определен оператор. Объявление и вызов функции `operator` осуществляется так же, как и любой другой функции. Единственное ее отличие заключается в том, что разрешается использовать сокращенную форму ее вызова. Так, выражение `operator#(a,b)`, можно записать в сокращенной форме `a#b`.

Рассмотрим пример. Программа доопределяет значение оператора «&». В результате ее можно будет использовать для выполнения операции пересечения множеств.

```
#include<iostream>
using namespace std;
class set                // класс «множество»
{
    char str[80];        // строка
public:
    set(const char *ss)   // это конструктор
    {
        strcpy_s(str,ss);
    }
    char * operator&(set); // объявление функции operator
};
```

```

char * set::operator&(set S) // описание функции operator
{
    int t=0, l=0;
    while(str[t++]!='\0'); // вычисление длины строки
    char *s1=new char[t]; // выделение памяти под строку
    for(int j=0; str[j]!='\0'; j++)
    {
        for(int k=0; S.str[k]!='\0'; k++)
        {
            if(str[j]==S.str[k])
            {
                s1[l]=str[j];
                l++;
                break;
            }
        }
    }
    s1[l]='\0';
    return s1;
}

int main()
{
    set S1="1f2bg5e6", S2="abcdef"; // задаются два множества
    cout << (S1 & S2) << endl; // результат fbe
    cout << (set("123") & set("426")) << endl; // результат 2
}

```

Ниже приведен пример программы перегрузки унарного оператора.

```

#include <iostream>
using namespace std;
class dek_koord
{
    int x,y; // декартовы координаты точки
public:
    dek_koord(){}; // конструктор без параметров
    dek_koord(int X,int Y) // конструктор с параметрами
    {
        x=X;
        y=Y;
    }
    void operator++(); // перегрузка префиксного оператора ++
    void operator++(int); // перегрузка постфиксного оператора ++
    dek_koord operator=(dek_koord); // перегрузка оператора =
    void see();
};

void dek_koord::operator++() // определение перегрузки оператора ++A
{
    x++;
}

```

```

}

void dek_koord::operator++(int)      // определение перегрузки оператора A++
{
    y++;
}
dek_koord dek_koord::operator =(dek_koord a)
{
    x=a.x;                          // определение перегрузки оператора =
    y=a.y;
    return *this;
}
void dek_koord::see()
{
    cout << "координата x = " << x << endl;
    cout << "координата y = " << y << endl;
}
int main()
{
    setlocale(LC_ALL,"Russian");
    dek_koord A(1,2), B, C; // объявление объектов
    A.see();
    A++;                    // увеличение значения компонента x объекта A
    A.see();                // просмотр содержимого объекта A
    ++A;                    // увеличение значения компонента y объекта A
    A.see();                // просмотр содержимого объекта A
    C=B=A;                  // множественное присваивание
    B.see();
    C.see();
    return 0;
}

```

Контрольные вопросы

1. Почему может потребоваться перегрузка оператора присваивания?
2. Можно ли изменить приоритет перегруженного оператора?
3. Когда следует переопределять операторы с помощью дружественных функций, а когда с помощью функций элементов класса?
4. Назовите особенности дружественных функций.
5. Опишите особенности перегрузки постфиксных и префиксных операторов «++» и «--».

Порядок выполнения работы

1. Изучить краткие теоретические сведения.
2. Ознакомиться с материалами литературных источников.
3. Ответить на контрольные вопросы.
4. Разработать алгоритм программы.
5. Написать, отладить и выполнить программу.

Задание

Расширить класс, написанный при выполнении лабораторной работы №1, добавив в него перегрузку функций, операторов + и &.

Написать класс для работы с символьными строками. Память под строку отводить динамически. Перегрузить операторы =, +, +=, ==, !=, [], ()(int, int). Продемонстрировать работу с этими операторами.

Литература

1. Шилдт, Г. Искусство программирования на C++ / пер. с англ. – СПб. : БХВ-Петербург, 2005. – 928 с.
2. Дейтел, Х. Как программировать на C++ / Х. Дейтел, П. Дейтел ; пер. с англ. – М. : Бином-Пресс, 2009. – 1037 с.
3. Страуструп, Б. Программирование. Принципы и практика использования C++ / Б. Страуструп ; пер. с англ. – М. : Вильямс, 2011. – 1246 с.
4. Страуструп, Б. Язык программирования C++. Специальное издание / Б. Страуструп ; пер. с англ. – М. : Вильямс, 2012. – 1136 с.
5. Буч, Г. Объектно-ориентированный анализ и проектирование с примерами приложений / Г. Буч ; пер. с англ. – М. : Вильямс, 2010 – 720 с.
6. Джамса, К. Учимся программировать на языке C++ / К. Джамса ; пер. с англ. – М. : Мир, 1997. – 320 с.
7. Павловская, Т. С/C++. Программирование на языке высокого уровня / Т. Павловская. – СПб. : Питер, 2013. – 464 с.
8. Луцик Ю.А. Объектно-ориентированное программирование на языке C++ : Учеб. Пособие / Ю.А. Луцик, В.Н. Комличенко. – Минск : БГУИР, 2008. – 266 с.